

15 — Python-Programme

Vom Skript zum strukturierten Programm

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Vom Skript zum Programm – warum Struktur?
2. Funktionen als Bausteine (Single Responsibility)
3. Die `main()`-Funktion – der Dirigent
4. Das `if __name__ == "__main__":`-Idiom
5. Die 5 Bausteine einer `.py`-Datei
6. Notebook vs. `.py`-Datei
7. Häufige Fehler vermeiden

Lernziel: Aus einzelnen Code-Schnipseln ein **richtiges Programm** machen – mit klarer Struktur, die auch ein Kollege versteht.

Wie wir heute arbeiten: Nach jedem Konzept zeigt die Folie *Live-Demo* (ich im Terminal) und *Sofort ausprobieren* (Sie im Notebook 15).

Motivation: Spaghetti oder Bauplan?

Ohne Struktur (“Spaghetti-Code”)

```
shop = {"Eco-Sneaker": 89.95, "Hemp-High": 109.00}
for p, preis in shop.items():
    print(f"{p}: {preis} EUR")
gesamt = 0
for preis in shop.values():
    gesamt = gesamt + preis
print(f"Summe: {gesamt} EUR")
```

- Alles auf einer Ebene – liest sich wie eine Wäscheliste
- Nicht wiederverwendbar, schwer testbar, schwer erweiterbar

Faustregel: Wird ein Skript länger als ca. 20 Zeilen, braucht es Struktur.

Motivation: Mit Struktur

```
def bestand_anzeigen(shop):  
    for p, preis in shop.items():  
        print(f"{p}: {preis} EUR")  
  
def gesamtwert(shop):  
    gesamt = 0  
    for preis in shop.values():  
        gesamt = gesamt + preis  
    return gesamt  
  
def main():  
    shop = {"Eco-Sneaker": 89.95, "Hemp-High": 109.00}  
    bestand_anzeigen(shop)  
    print(f"Summe: {gesamtwert(shop)} EUR")
```

Funktionen als Bausteine

Single Responsibility: eine Funktion, eine Aufgabe

Schlecht

`daten_laden_und_speichern()`
`berechne_und_zeige()`
`pruefen_und_senden()`

Besser

`daten_laden() + daten_speichern()`
`berechne() + zeige()`
`pruefen() + senden()`

Test: Wenn Sie die Funktion mit “und” beschreiben müssen, sind es vermutlich **zwei** Funktionen.

 **Sofort ausprobieren** – Notebook 15, Kap. 2: Übung 2.1
(durchschnitt_berechnen, beste_note_finden, bestanden_zaehlen)

Funktionen rufen Funktionen auf

```
def bericht_erstellen(shop):  
    """Erstellt einen Lagerbericht."""  
    print("=== Veggie Soles Bericht ===")  
    bestand_anzeigen(shop)  
    summe = gesamtwert(shop)  
    print(f"Lagerwert: {summe:.2f} EUR")
```

- bericht_erstellen muss **nicht wissen**, wie der Bestand angezeigt wird
- Sie delegiert – die Details stecken in bestand_anzeigen
- Das nennt man **Abstraktion**: Schichten von Bedeutung

► **Live-Demo** - 02_funktionen_als_bausteine.py



Die main()-Funktion - der Dirigent

```
def main():  
    """Hauptprogramm: Veggie Soles Shop."""  
    shop = {"Eco-Sneaker": 89.95, "Hemp-High": 109.00,  
            "Bambus-Boot": 135.50}  
    bestand_anzeigen(shop)  
    print(f"Lagerwert: {gesamtwert(shop):.2f} EUR")
```

main()

- **Steuert den Ablauf** – ruft Hilfsfunktionen in der richtigen Reihenfolge auf
- **Enthält selbst wenig Logik** – die steckt in den Hilfsfunktionen
- **Liest sich wie eine Zusammenfassung** Ihres Programms

Was gehört in `main()` - und was nicht?

Gehört in <code>main()</code>	Gehört NICHT in <code>main()</code>
Programmablauf steuern	Funktionsdefinitionen
Hilfsfunktionen aufrufen	Import-Anweisungen
Anfangsdaten bereitstellen	Konstanten-Definitionen
Benutzereingaben (wenn nötig)	Komplexe Berechnungslogik
Ergebnisse ausgeben	Hilfsfunktionen

Test: Wer nur `main()` liest, versteht **was** das Programm tut – ohne **wie**.

 **Sofort ausprobieren** – Notebook 15, Kap. 3: Übung 3.1 (`main()` für Notenauswertung), Übung 3.2 (Refaktorisieren)

Das `if __name__ == "__main__":`-Idiom

```
# shop_utils.py
```

```
def bestand_anzeigen(shop):  
    for p, preis in shop.items():  
        print(f"{p}: {preis} EUR")
```

```
def main():  
    bestand_anzeigen({"Eco-Sneaker": 89.95})
```

```
if __name__ == "__main__":  
    main()
```

- `__name__` ist eine **eingebaute Variable**
- Direkter Aufruf (`python shop_utils.py`): `__name__ == "__main__"`
→ `main()` läuft
- Import (`import shop_utils`): `__name__ == "shop_utils" > main()`

Skript vs. Modul - eine Datei, zwei Rollen

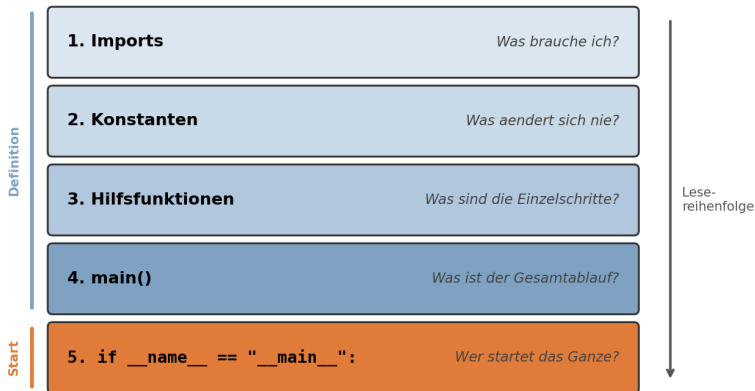
Aspekt	Skript (direkt ausgeführt)	Modul (importiert)
<code>__name__</code>	<code>"__main__"</code>	Dateiname (z. B. <code>"shop_utils"</code>)
Zweck	Programm starten	Funktionen bereitstellen
Beispiel	<code>python shop.py</code>	<code>import shop_utils</code>

Mit `if __name__ == "__main__":` kann **dieselbe Datei beides sein** - Skript *und* importierbares Modul.

► **Live-Demo** - `04_modul_oder_skript.py`

✎ **Sofort ausprobieren** - Notebook 15, Kap. 4: Übung 4.1 (`__name__` untersuchen), Übung 4.2 (Modul-Verhalten simulieren)

Die 5 Bausteine einer .py-Datei



Reihenfolge ist **nicht zufällig**: Was die Funktionen brauchen (Imports, Konstanten) steht oben, was sie startet, ganz unten.

Vollständiges Beispiel - Veggie Soles

```
# --- 1. Imports ---  
# (keine)  
  
# --- 2. Konstanten ---  
VERSANDPAUSCHALE = 4.95  
FREI_AB = 100.00  
  
# --- 3. Hilfsfunktionen ---  
def versand_berechnen(summe):  
    if summe >= FREI_AB:  
        return 0.0  
    return VERSANDPAUSCHALE  
  
# --- 4. main() ---
```

Notebook vs. .py-Datei

Aspekt	Notebook	.py-Datei
Ausführung	Zellen, beliebige Reihenfolge	Sequenziell von oben
Zustand	Bleibt zwischen Zellen	Frisch bei jedem Start
<code>__name__</code>	Immer <code>"__main__"</code>	<code>"__main__"</code> oder Modulname
Ideal für	Lernen, Experimentieren	Programme, Tools, Module
Versionierung	Schwierig (JSON)	Einfach (reiner Text)

Das Notebook ist Ihre Werkbank - die .py-Datei ist das fertige Produkt.

Im Notebook: `main()` direkt aufrufen. In der .py-Datei: `if __name__ == "__main__": main()`.

Anti-Patterns - was Sie vermeiden

Anti-Pattern	Warum schlecht?
Loser Code statt Funktion	Beim Import würde alles sofort laufen
Globale Variablen statt Parameter	Versteckte Abhängigkeiten, nicht testbar
Riesige <code>tue_alles()</code> -Funktion	Unlesbar, nicht wiederverwendbar
<code>print()</code> statt <code>return</code> in Berechnungen	Aufrufer kann Ergebnis nicht weiterverwenden
Kryptische Namen (<code>bp()</code> , <code>f3()</code>)	Niemand versteht den Code in 2 Wochen
Konstanten klein geschrieben	Verschwimmen mit Variablen

 **Sofort ausprobieren** – Notebook 15, Kap. 7: Übung 7.1 (“Vier Strukturfehler finden”)

Cheat Card

Konzept	Syntax
Funktion definieren	<code>def name(parameter):</code>
Hauptfunktion	<code>def main():</code>
Einstiegspunkt	<code>if __name__ == "__main__":</code>
Konstante	<code>MAX_WERT = 100</code>
Docstring	<code>"""Beschreibung."""</code>
Wert zurückgeben	<code>return wert</code>

Die 5 Bausteine: Imports → Konstanten → Hilfsfunktionen → main() → if __name__.

Ausblick: Notebook 16 - Pandas

- **DataFrames:** tabellarische Daten elegant verarbeiten
- **Visualisierungen:** Diagramme direkt aus Daten
- **CSV-Dateien:** echte Datensätze laden
- **Externe Bibliotheken:** `import pandas as pd`

Auch Datenanalyse-Programme profitieren von `main()` und guter Struktur – die heutigen Bausteine bleiben Ihr Werkzeug.

Heute geübt

- ✓ Spaghetti-Code von strukturiertem Programm unterschieden
- ✓ Funktionen nach **Single Responsibility** entworfen
- ✓ Funktionen rufen Funktionen auf (Abstraktion)
- ✓ `main()` als Dirigent geschrieben
- ✓ `if __name__ == "__main__":` verstanden
- ✓ Die **5 Bausteine** einer `.py`-Datei kennengelernt
- ✓ Häufige Anti-Patterns erkannt

Zur Vertiefung im Notebook 15:

- “Sofort ausprobieren“-Aufgaben in Kap. 1-7 (sind Sie schon mitgegangen)
- Abschlussübungen: 4 Aufgaben (Taschenrechner, Notenrechner, Lagerverwaltung, Lohnabrechnung refaktorisieren)